# Plan Técnico 08: Fotogrametría RGB-D y Calibración Automática en Tiempo Real

> **ESTADO DEL DOCUMENTO:** Plan de Arquitectura e Investigación Técnica
> **Fecha:** Octubre 2026 | **Área:** Visión por Computador, Fotogrametría y Fusión Multi-Cámara
> **Hardware Objetivo:** 2x Intel RealSense D455 (RGB + Stereo IR Activo)

---

## 1. ¿Cómo Funciona la Fotogrametría?

La **fotogrametría** es la ciencia y disciplina tecnológica que permite reconstruir geometría tridimensional métrica y texturas 3D a partir de colecciones de imágenes fotográficas bidimensionales superpuestas.

**Pipeline Clásico de Fotogrametría (SfM + MVS):**

```
[ Fotos 2D ]
     ■
     ▼
[ 1. Detección de Features ] ■■■ SIFT, ORB, SuperPoint (puntos clave invariantes)
     ■
     ▼
[ 2. Feature Matching ]      ■■■ Emparejamiento por descriptores (FLANN / Ratio Test)
     ■
     ▼
[ 3. Structure from Motion ] ■■■ Matriz Esencial (E), Geometría Epipolar, Poses R|t
     ■
     ▼
[ 4. Bundle Adjustment ]     ■■■ Optimización no lineal (minimiza error de reproyección)
     ■
     ▼
[ 5. Multi-View Stereo ]     ■■■ Densificación píxel a píxel (Mapas de profundidad MVS)
     ■
     ▼
[ 6. Mallado y Texturizado ] ■■■ Poisson Surface Reconstruction / Delaunay + Proyección UV
```

**Limitación Intrínseca de la Fotogrametría Clásica:**

1. **Ambigüedad de Escala (*Scale Ambiguity*):** Con cámaras fotográficas convencionales (RGB monocular), la traslación relativa $\mathbf{t}$ solo se conoce como un vector unitario de dirección, no en metros reales. Un objeto de 10 cm fotografiado a 30 cm produce exactamente los mismos rayos visuales que un edificio de 10 metros fotografiado a 30 metros.
2. **Costo Computacional Extremo:** La etapa de densificación estéreo multi-vista (MVS) y el ajuste de haces (*Bundle Adjustment*) requieren comparar millones de parches de píxeles, tardando minutos u horas en software como Meshroom, RealityCapture o COLMAP.

## 2. ¿Hacemos Algo Relacionado en Este Proyecto?

**Sí. Nuestro proyecto implementa un puente híbrido entre Fotogrametría y Sensado Activo de Profundidad:**

| Componente del Proyecto | Técnica Fotogramétrica Implementada | Biblioteca / Algoritmo |
|---|---|---|
| **Mapeo Multi-Foto (`multiscan.py`)** | Extracción de descriptores invariantes locales 3D | FPFH (*Fast Point Feature Histograms*) |
| **Mapeo Multi-Foto (`multiscan.py`)** | Emparejamiento global de poses sin estimación previa | RANSAC basado en correspondencia de características |
| **Mapeo Multi-Foto (`multiscan.py`)** | Ajuste fotométrico denso por gradientes de color RGB | *Colored ICP* (Park et al.) |
| **Mapeo Multi-Foto (`multiscan.py`)** | Optimización global de trayectoria de captura | *Pose Graph Optimization* (cierre de bucle 360°) |
| **Captura Dual (`capture.py`)** | Reconstrucción de superficie continua e interpolación | *Poisson Surface Reconstruction* (`depth=8`) |
| **Live 3D (`shaders.js`)** | Proyección epipolar y remapeo UV en GPU | Matrices intrínsecas pinhole ($f_x, f_y, c_x, c_y$) |
| **Blender Addon (`script_blender.py`)** | Reconstrucción de malla con textura fotográfica | Mapeo de coordenadas UV y proyección de vértices |

### La Ventaja Tecnológica de RealSense:

A diferencia de la fotogrametría tradicional donde la profundidad debe calcularse conjeturando correspondencias fotométricas costosas, el sensor **Intel RealSense D455 proyecta un patrón láser infrarrojo estructurado y calcula la profundidad densa directamente en silicio ASIC (chip D4) a 30-60 FPS**.

Esto nos permite saltarnos el 90% del procesamiento pesado de la fotogrametría clásica y operar a tasas interactivas en tiempo real.

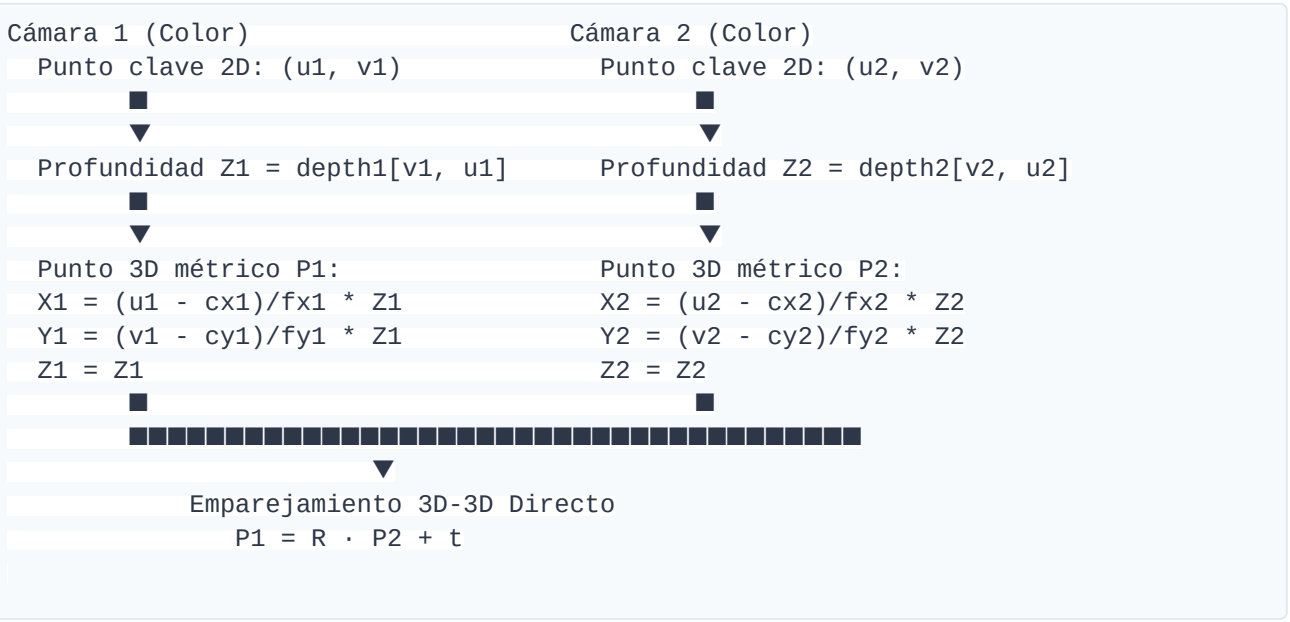
---

## 3. Fotogrametría RGB-D en Tiempo Real: Match Automático de Cámaras

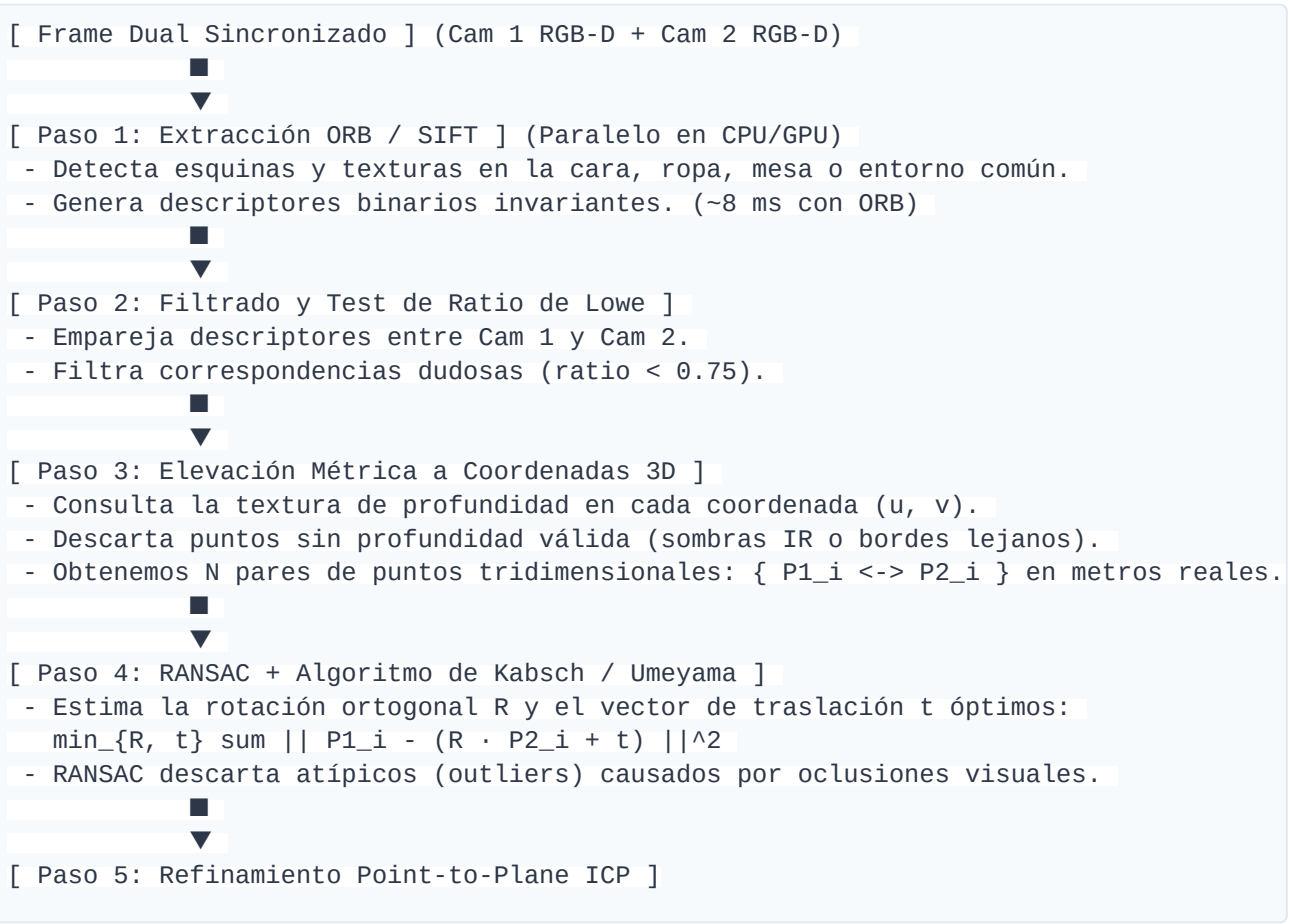
Para eliminar completamente la necesidad de calibraciones manuales con sliders o tableros impresos ChArUco, es posible implementar un **Auto-Matcher Fotogramétrico RGB-D**.

### Fundamento Matemático: Elevación 2D $\to$ 3D (RGB-D Feature Lifting)

En lugar de resolver la geometría epipolar pasiva con ambigüedad de escala, aprovechamos que **cada píxel de color $(u, v)$ tiene su profundidad métrica $Z$ ya medida por el sensor**:




### Algoritmo Paso a Paso:



```
 - Utiliza la solución de Kabsch como estimación inicial (semilla perfecta).
 - Itera 15 pasos de ICP sobre la geometría densa para precisión sub-milimétrica.
                  ■
                  ▼
[ Matriz T(2->1) Calibrada ] ■■■ Actualización en caliente en Three.js y config JSON.
```

---

## 4. Comparativa de Tecnologías para el Match Automático

| Método | Tipo | Velocidad | Precisión | Requiere Tablero | Fortaleza | Debilidad |
|---|---|---|---|---|---|---|
| **ChArUco Board** | Calibración Óptica | ~500 ms | < 1 mm | **Sí** | Máxima precisión matemática | Incómodo, requiere imprimir tablero |
| **Sliders Manuales** | Visual Humano | Interactiva (60 FPS) | 5 - 15 mm | No | Rápido para ajustes intuitivos | Depende del ojo del operador |
| **ORB + Kabsch 3D** | Fotogrametría RGB-D | **~15 - 30 ms** | 2 - 5 mm | **No** | Tiempo real continuo sin tableros | Sensible a superficies sin textura |
| **SIFT + RANSAC 3D** | Fotogrametría RGB-D | ~80 - 150 ms | 1 - 3 mm | **No** | Robusto a grandes ángulos (> 45°) | Mayor consumo de CPU que ORB |
| **SuperPoint + LightGlue** | Deep Learning | ~40 ms (GPU) | < 2 mm | **No** | Inmune a cambios extremos de luz | Requiere modelo neuronal (ONNX/Torch) |

---

## 5. Diseño de Implementación en el Proyecto

### 5.1 Nuevo Módulo: `live3d/feature_matcher.py`

Implementará la clase `RGBDFeatureMatcher`:

```
class RGBDFeatureMatcher:
    def __init__(self, method="ORB", max_features=1500):
        # ORB para tiempo real (<25ms) o SIFT para robustez angular amplia
        if method == "ORB":
            self.detector = cv2.ORB_create(nfeatures=max_features)
            self.matcher = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=False)
        else:
            self.detector = cv2.SIFT_create(nfeatures=max_features)
            self.matcher = cv2.BFMatcher(cv2.NORM_L2, crossCheck=False)

    def match_and_solve_pose(self, snap1, snap2, min_matches=6):
        """
        Calcula T(2->1) directamente a partir de las imágenes de color y profundidad.
        Devuelve: (T_matrix, inliers_count, rmse_mm, debug_canvas)
        """
        ...
```

### 5.2 Modos de Operación:

1. **Modo 1-Clic: "■ Match Fotogramétrico Automático"**
2. El usuario simplemente se sienta frente a las cámaras (o coloca el objeto a escanear) y pulsa el botón.
3. El sistema busca puntos clave en su cara, camiseta o entorno, calcula la pose tridimensional mediante Kabsch-RANSAC, refina con ICP y actualiza la vista inmediatamente.
4. **Modo Continuo de Fondo (Auto-Alineación Dinámica):**
5. Si las cámaras se mueven accidentalmente o están en un soporte flexible, un hilo de fondo ejecuta el matcher cada 3 segundos.
6. Si detecta un cambio geométrico significativo con alta confianza, re-calibra la pose suavemente (*lerp* temporal) sin interrumpir la transmisión en vivo.

---

## 6. Fases de Ejecución Técnica

- [ ] **Fase 1: Módulo de Detección y Elevación 3D ( `live3d/feature_matcher.py` )**
- Implementar extracción de características 2D (ORB y SIFT).
- Proyección de coordenadas 2D a espacio métrico 3D con intrínsecas pinhole.
- Estimación rígida cerrada de Kabsch/Umeyama con RANSAC.
- [ ] **Fase 2: Endpoint REST en el Servidor ( `live3d/server.py` )**
- Añadir ruta `POST /api/dual/photogrammetry_match`.
- Soporte de retorno de imagen de depuración (con líneas visuales conectando los puntos emparejados).

- [ ] **Fase 3: Integración en la Interfaz Web (`index.html` y `main.js`)**
- Botón: `■ Match Fotogramétrico (Sin Tablero)`.
- Visualización opcional de los puntos clave detectados en pantalla (*overlay* 2D).
- [ ] **Fase 4: Validación y Pruebas con Sujetos Reales**
- Comprobación en diferentes condiciones de iluminación y ropa con/sin textura.
- Validación del error residual frente a la calibración ChArUco (debe ser $< 8\text{ mm}$).